

Building your own text-generating AI (4/4)

Lecture 11 – Tokenisation, sampling, and what you've built

Dr Thomas Robinson

August 18, 2026

Warm-up · three questions from last time

≈ 5 MIN · SHOUT IT OUT

1. In attention, what are the roles of the **query**, the **key**, and the **value** — one phrase each?
2. Why do we divide the attention scores by $\sqrt{d}$?
3. What does the **causal mask** block, and why do we need it?

Warm-up · answers

1. **Query**: what I am looking for. **Key**: what I contain. **Value**: what I hand over if matched.
2. Dot products grow with dimension. Unscaled, the scores saturate the softmax and one token takes nearly all the weight. Dividing by $\sqrt{d}$ keeps the scores in a range where the softmax still spreads attention.
3. It blocks attention to **future tokens**. Without it, next-token prediction could read the answer it is supposed to predict.

| Part 1 · Tokenisation

Three levels of tokenisation

Character

- Tiny vocab (~100).
- Sequences are **long** — typically 4× longer than word-level.
- Model spends compute learning spelling.
- What our labs have used since Lab 8.

Word

- Huge vocab (10^5+).
- OOV (out-of-vocabulary) failure mode.
- Embedding matrix is gigantic.
- What 1990s/2000s NLP used.

Subword

- Vocab ~32k–128k.
- No OOV — rare words just split into smaller pieces.
- What every modern LLM uses.
- The topic of this lecture.

Where compute goes — and why tokenisation matters

For a transformer with width d at sequence length T — yesterday's T , one position per token:

- Self-attention cost per layer: $\mathcal{O}(T^2 d)$.
- Feed-forward cost: $\mathcal{O}(T d^2)$.
- Memory for the attention matrix: $\mathcal{O}(T^2)$.



Tip

Longer sequences are quadratically more expensive.

Halving the average tokens-per-word — by better tokenisation — halves the sequence length. And **quarters the attention compute**.

Tokenisation choices have downstream cost implications.

Token cost varies by language

THE HIDDEN COST OF BEING NON-ENGLISH

“Hello, how are you today?” in GPT-4’s tokeniser:

- **English:** 7 tokens.
- **Spanish:** 8 tokens.
- **Chinese:** 10 tokens.
- **Burmese:** 71 tokens.
- **Tamil:** 49 tokens.

Same content. **Ten times the cost** for some languages.

This is a structural inequity. API pricing is per token. Context windows are per token. A Burmese-speaking user pays ten times more *and* fits one-tenth the conversation.

Petrov et al. (2023) — *Language Model Tokenizers Introduce Unfairness Between Languages*.

Tokenisation's strangest bugs

SURPRISES YOU'LL MEET IN THE WILD

Numbers

GPT-4 tokenises numbers **inconsistently**:

- Some years are a **single token**.
- An *adjacent* year can split into **several** tokens.
- There's no clean rule — it depends on what was frequent in the training data.

A model that sees `2024` as one atom but `2025` as three pieces has no stable notion of "a year"

- This inconsistency is part of why LLMs are sometimes **bad at arithmetic**

Glitch tokens

Some tokens appear in the vocabulary but rarely in training data. The model has no idea what they mean.

- "`SolidGoldMagikarp`" produced bizarre, evasive output in GPT-2/GPT-3-era models.
- Reddit-derived artefacts; the vocabulary outlived the user.



The lesson

Vocabulary is a one-shot decision made before training. Mistakes persist for the model's lifespan. Modern tokenisers (cl100k, o200k) try harder to avoid these.

How many r's in strawberry?

THE FAILURE WE SAW AT THE BEGINNING OF THE COURSE

Ask a frontier model to count the r's in "strawberry" and it has, famously, answered **two**.

What the model actually receives

Not `s t r a w b e r r y`.

Something closer to `str aw berry` — three tokens, three vectors.

The letters were gone before the model saw anything.
There is no `r` in its input to count.

So it answers from memory

Spelling a token means recalling which characters it *stood for*, which is a fact about the tokeniser, not about the text.

The model answers from what it has read *about* spelling, rather than by looking.



The model reasons over tokens, not characters or digits.

It is why spacing the word out — `s t r a w b e r r y` — often fixes it, why "write code to count them" fixes it every time, and why a reasoning model that spells the word out first gets there too.

What is a byte?

THE UNIT EVERYTHING ELSE IS BUILT FROM

A **byte** is 8 bits. Each bit is 0 or 1, so a byte has $2^8 = 256$ possible values — the whole numbers 0 to 255.

Text is stored as numbers

A computer has no notion of a letter. It stores a number, and an **encoding** says which number means which character. The standard is UTF-8.

Character	Stored as
A	65
a	97
!	33
space	32



Note

Not every character is one byte — **é** takes two, an emoji takes four. We'll come back to that once BPE is running.

The alphabet is only 256 long

So **low** reaches the computer as **108 111 119**.

Your name, this sentence, every training corpus ever scraped — all of it is a sequence of numbers from 0 to 255.

There are 256 of them, and there will never be a 257th.

Byte-pair encoding — the standard

THE BPE ALGORITHM — START WITH BYTES

Initialise: every byte is a token. Vocab size 256.

Repeat until target vocab size (e.g. 50,000):

1. Count all adjacent token **pairs** in the corpus.
2. Find the most frequent pair.
3. Merge it into a new token.
4. Add the new token to the vocabulary.



Note

That's the whole algorithm. It was invented for data compression in the 1990s. Sennrich, Haddow & Birch (2016) applied it to neural machine translation. It is now the default tokeniser in every major LLM.

A worked BPE example

TINY CORPUS

low low low low low lowest newer newer newer newer newer wider

Initial state

Single characters. Each box is **one token** (`_` marks the word end):

- `l` `o` `w` `_` ×5
- `l` `o` `w` `e` `s` `t` `_` ×1
- `n` `e` `w` `e` `r` `_` ×5
- `w` `i` `d` `e` `r` `_` ×1

Most-frequent pair counts

- `(e,r)` · `(l,o)` · `(o,w)` · `(r,_)` · `(w,e)` — **6 each**
- `(n,e)` · `(e,w)` · `(w,_)` — 5 each

Five pairs tie at the top. Break ties by a fixed rule; here we take `(e,r)`.

Merge `(e,r)` → new token `er`. Repeat.

BPE merges — step by step

Each row applies **exactly one** merge. Every box is one token, and a box like **er** is the token that row's merge just created. Read **down** a column to watch one word collapse:

Step	Merge	low ×5	lowest	newer ×5	wider
0	(start)	l o w _	l o w e s t _	n e w e r _	w i d e r _
1	(e,r)→er	l o w _	l o w e s t _	n e w er _	w i d er _
2	(er,_)→er_	l o w _	l o w e s t _	n e w er_	w i d er_
3	(l,o)→lo	lo w _	lo w e s t _	n e w er_	w i d er_
4	(lo,w)→low	low _	low e s t _	n e w er_	w i d er_
5	(low,_)→low_	low_	low e s t _	n e w er_	w i d er_

Four boxes down to one: after five merges the frequent word **low_** is a **single token**. Keep going and **newer_**, **wider_** collapse too.

The rare word **lowest_** stays split — five boxes — but the first one is **low**, the **same piece** the common word uses. That sharing is the point: the model can spell a word it has barely seen out of pieces it knows well. **No word is ever out-of-vocabulary.**

How modern tokenisers look

GPT-2

Byte-level BPE — operates on raw utf-8 bytes. Any text works.

Vocab: 50,257.

GPT-4 / CLAUDE

`cl100k_base` and variants — larger vocabularies, refined merge ordering for code + multilingual text.

Vocab: ~100,000+.



Note

Try [OpenAI's tokeniser visualiser](#). Type a sentence in English, then in another language. Watch the token count change.

Byte-level BPE — the missing detail

Our worked examples merged **characters**. But this won't work for every possible word in our vocabulary:

Character-level BPE

- Base vocabulary = the characters seen in training.
- Unicode has ~150,000 characters. No corpus contains them all.
- Meet a new one at test time — a rare script, a fresh emoji — and the tokeniser has no token for it (and so gets assigned `<unk>`).

Byte-level BPE — GPT-2's fix

- Run BPE on **bytes**, not characters.
- Base vocabulary = **all 256 byte values**.
- Every string a computer stores is a byte sequence, so every possible string is representable.
- There is no unknown token, ever.

Bytes in practice — one character is not one byte

In UTF-8, common Latin characters take one byte each; everything else takes more:

- `café` → `c`, `a`, `f` at one byte each, `é` at **two** (`0xC3 0xA9`): $3 \times 1 + 2 = 5$ bytes.
- 🤖 → **four** bytes (`0xF0 0x9F 0xA4 0x96`).

If 🤖 is rare in the training corpus, no merge ever forms it

- The tokeniser hands the model **four single-byte tokens**
- The model has to piece the robot together itself

TRY IT YOURSELF

Why can byte-level BPE never meet an out-of-vocabulary string?

ANSWER

Every string is stored as a sequence of bytes, and all 256 possible byte values are in the vocabulary. Worst case, a string tokenises byte by byte — but it always tokenises.

| Plenary · BPE by hand

Plenary · BPE by hand

≈ 10 MIN · IN PAIRS

TINY CORPUS

"banana banana banana ana ana"

Use a `_` to mark word ends. Each character + `_` starts as a token.

TASK

Do **three** BPE merges by hand. After each, write down:

1. Which pair you merged.
2. The current state of the corpus.
3. The current vocabulary.

REFLECTION

After your three merges, would "banana" be a single token or several? What about "bandana" (a word the corpus has never seen)?

Six minutes.

Plenary · answers

START "banana" = `b a n a n a _` (×3) · "ana" = `a n a _` (×2)

INITIAL PAIR COUNTS `(a,n)` 8 · `(n,a)` 8 · `(a,_)` 5 · `(b,a)` 3.

STEP 1 `(a,n)` and `(n,a)` tie at 8 — break ties by a fixed rule; here take `(n,a) → na`.

- "banana" → `b a na na _`
- "ana" → `a na _`

STEP 2 Recount: `(a,na)` and `(na,_)` now tie at 5 (top) — fixed rule again takes `(a,na) → ana`.

- "banana" → `b ana na _`
- "ana" → `ana _`

STEP 3 Recount: `(b,ana)`, `(ana,na)`, `(na,_)` all tie at 3 — fixed rule picks `(b,ana) → bana`.

- "banana" → `bana na _`
- "ana" → `ana _`

REFLECTION

"**banana**" now tokenises as `bana na _` — three boxes, down from seven.

"**bandana**" starts as `b a n d a n a _`. Apply the *learned* merges in order — `na`, then `ana`, then `bana`:

- `na`: only the final `n a` merges (the first `n` is followed by `d`) → `b a n d a na _`
- `ana`: the `a na` merges → `b a n d ana _`
- `bana`: needs `b` next to `ana`, but `b` is followed by `a` → no merge.

Result: `b a n d ana _`. The model has *never seen* "bandana" yet reads it from known pieces.

This is BPE's guarantee: **no OOV**.

| Part 2 · Sampling strategies

We always sample — but *how*?

At each step of generation, the model gives us a distribution p over the next token. To pick one:

ARGMAX (GREEDY)

Always pick the most likely.

- Deterministic.
- Often repetitive / dull.
- Bad for chat.

Sample from p

Sample with original probabilities.

- Diverse.
- Sometimes wanders into low-probability nonsense.

Beam search

Track top- B candidate sequences in parallel.

- Useful for translation.
- Boring for chat.

Temperature

Adjust the softmax with a **temperature** $\tau > 0$:

$$p_i \propto \exp\left(\frac{z_i}{\tau}\right)$$

$$\tau < 1$$

Sharpen the distribution. More confident, more deterministic.

$$\tau > 1$$

Flatten the distribution. More creative, more random.



Tip

$\tau = 0$ reduces to argmax. $\tau = 1$ keeps the raw model probabilities. Most chat applications run somewhere in 0.7–1.0.

For technical/factual tasks, lower (≈ 0.2). For creative writing, higher (~ 1.2).

Beam search — for translation, not chat

THE “FIND THE BEST WHOLE SENTENCE” APPROACH

Argmax picks one token at a time — locally optimal, globally myopic.

Beam search keeps the top B candidate *sequences* in parallel, extending each. At the end, return the highest-scoring sequence overall. B is the **beam width**.

- $B = 5$ or 10 is common.
- Used for translation, summarisation — anywhere there’s a “single best answer.”

Why not for chat. Beam search produces fluent but predictable text.

- The “best” sequence is often generic
- For chat, conversation, or creative writing, sampling is better (feels more human)

Rule of thumb: beam search for one-shot tasks with a verifiable correct answer. Use sampling for open-ended generation.

What does beam search cost?

SEARCHING HARDER IS NOT FREE

A beam width of B means B live sequences. Each one needs its **own forward pass at every step**.

Per generated token

- B forward passes, not 1.
- B KV caches held in memory — yesterday's cache, once per beam.
- Score all $B \times V$ continuations, keep the best B .

With $V \approx 50,000$ and $B = 5$, that last step sorts 250,000 numbers. Nothing, next to five passes through the network.

The bill

- $B = 5 \rightarrow$ **5× the compute** and **5× the KV-cache memory**, for every token generated. The weights are shared, so they are not multiplied.
- The B passes batch together, so on an idle GPU the wall-clock is kinder than 5×. The FLOPs and the memory bandwidth are not.
- When you are serving users, the batch is already full. Then it is a straight **5× cut in throughput**.



Tip

Translation pays this happily — outputs are short and there is one right answer. A chat model would pay it on every token of every reply, forever, and get *more* generic text for the money. Holtzman et al. (2019) made exactly that argument, and proposed top-p instead. That's the next slide.

Top-k and top-p (nucleus)

Both cut tokens out of the distribution, then **renormalise** — divide by what's left, so the survivors sum to 1 again — and sample from what remains.

Top-k

Keep only the k highest-probability tokens; renormalise; sample.

- Truncates the long tail of garbage.
- $k = 40$ or 50 common.
- Same k regardless of model confidence.

Top-p (nucleus)

Keep the smallest set of tokens whose cumulative probability exceeds p ; renormalise; sample.

- **Adapts** to model confidence.
- $p = 0.9$ common.
- Few tokens kept when model is confident; many when uncertain.



Note

Combined with temperature, we have four hyperparameters on creativity:

1. Argmax / sampling switch
2. Temperature
3. Top-k
4. Top-p

In practice, GPT-style chat models use a low temperature + top-p ≈ 0.9 .

When top-k and top-p disagree

Two next-token distributions over the same six tokens — one from a confident model, one from an uncertain one:

Peaked

$$p = [0.85, 0.06, 0.04, 0.03, 0.01, 0.01]$$

- **Top-p**, $p = 0.7$: the first token alone gives $0.85 \geq 0.7 \rightarrow$ keeps **1** token.
- **Top-k**, $k = 3$: keeps **3** tokens — including two the model rates at 6% and 4%.

Flat

$$p = [0.25, 0.20, 0.20, 0.15, 0.10, 0.10]$$

- **Top-p**, $p = 0.7$: $0.25 + 0.20 + 0.20 = 0.65 < 0.7$, so add the fourth $\rightarrow$ keeps **4** tokens.
- **Top-k**, $k = 3$: keeps **3** — cutting a token the model rates at 15%.

Same settings, both times. Top-p kept 1 token in one case and 4 in the other.



Tip

The difference in one line: **top-p adapts the cut to the model's confidence; top-k cannot**. When the model is sure, top-p gets out of the way. When it is unsure, top-p lets more candidates through.

After the cut — renormalise

Whatever survives the cut must sum to 1 again before we sample. Take the flat distribution under top-k, $k = 3$:

$$\begin{aligned}\text{kept} &= [0.25, 0.20, 0.20], & \text{sum} &= 0.65 \\ \text{renormalised} &= \left[\frac{0.25}{0.65}, \frac{0.20}{0.65}, \frac{0.20}{0.65} \right] \approx [0.385, 0.308, 0.308]\end{aligned}$$

Then sample from the renormalised distribution as usual.

How does generation know when to stop?

WHEN SHOULD WE STOP PICKING?

The EOS token

The vocabulary holds a token meaning *"the text ends here"*. GPT-2 writes it `<|endoftext|>`.

- It is **added by hand**, not built by BPE merges — no byte sequence spells it.
- The model is trained to predict it, so it is one more entry in the softmax.
- Sample it, and generation stops.

The two blunter tools

- `max_tokens` — a hard ceiling. Your lab sampler's `max_new_tokens` is this, and only this.
- **Stop sequences** — the caller names a string (`"\n\n"`, `"User:"`); generation halts when it appears.

So a reply ends one of three ways: the model chose to, your budget ran out, or a string matched.



Special tokens are also what makes a chat model

The same trick wraps every turn of a conversation in markers the model was trained to recognise — a **chat template**. It is why an API takes a list of role-tagged messages rather than one string: the tokeniser turns those roles into special tokens before the model reads a word.

Back to text — decoding

THE RETURN TRIP

Our sampler emits token IDs. Turning them back into a string is a lookup (with one trap)

The lookup

Each token ID maps to a **byte sequence**. Concatenate the bytes for the whole output, then decode once as UTF-8.

low er → 108 111 119 101 114 → "lower".

The trap

Don't decode token by token. One character can span several tokens — é is two bytes, 🤖 is four, and no merge need ever have joined them.

Decode each token alone and a multi-byte character arrives broken — Python hands you 🤖, the replacement character.

- You will meet this in the lab if you print while you sample.



Tip

Bytes in, bytes out. The byte sequence is what round-trips; token boundaries are an implementation detail the text does not respect.

| **Activity · sampling strategies**

Activity · predict the output

≈ 8 MIN · IN PAIRS

We give a trained GPT the same 10-token prompt and generate 50 tokens with each sampling strategy:

- A. Greedy / argmax ($\tau = 0$).
- B. Temperature $\tau = 0.7$, top-p = 0.9.
- C. Temperature $\tau = 1.5$, no top-k/top-p.
- D. Temperature $\tau = 0.7$, top-k = 1.

PREDICT EACH OUTPUT'S CHARACTER — CHOOSE FROM:

- i. Coherent and slightly varied.
- ii. Deterministic; often repeats a phrase.
- iii. Highly varied; sometimes nonsense.
- iv. Slightly varied at the start, more chaotic later.

BONUS: which two are functionally **the same**?

Five minutes.

Activity · answers

A: **(ii)** — argmax is deterministic; can repeat (“loop”).

B: **(i)** — moderate τ + top-p is the modern chat default.

C: **(iii)** — high temperature with no truncation lets the tail dominate.

D: **(ii)** — top-k=1 collapses sampling to argmax regardless of temperature.

A and D are functionally the same. Both are deterministic.



Tip

Sampling is **interface design** for LLMs. Different products tune these for different feels — terse + factual, or rambling + creative. The same model produces different “personalities” depending on these knobs.

| **Part 3 · Beyond a base GPT**

A reflection on what you've built

The reveal — sort of

By end of tomorrow's lab, your tiny GPT will have:

- A proper **tokeniser** (BPE).
- A **token embedding** matrix.
- A **positional embedding**.
- N **transformer blocks** with multi-head causal attention.
- A **softmax + sampler** producing one token at a time.

Same skeleton as GPT-4, Claude, Gemini. The differences are scale (you: millions of parameters; them: trillions), data, and post-training.

You are no longer treating these models as black boxes.

Pre-training

A FRONTIER MODEL'S FULL LIFECYCLE

1. Pre-training

What we have built so far.

- Predict the next token over trillions of tokens of web text + books + code.
- Output: a “base model” — fluent, knowledgeable, useless as a chat assistant.
- Cost: tens of millions of dollars. Months of compute.

Post-training

A FRONTIER MODEL'S FULL LIFECYCLE (CONTINUED)

2. SUPERVISED FINE-TUNING (SFT) — "INSTRUCTION TUNING"

Continue training on (prompt, ideal-response) pairs. One example, role tokens included:

`<|user|>` Explain a p-value in two sentences. `<|assistant|>` A p-value is the probability of seeing data at least as extreme as yours if the null hypothesis were true. It is not the probability that the null is true. `<|end|>`

- Output: a model that follows instructions.
- Where the pairs come from: paid writers; licensed data; model-generated drafts filtered by humans.
- Cost: ~10,000 to ~100,000 examples; a few days of compute.

3. Preference tuning (RLHF / DPO)

Train the model to prefer responses that humans rate higher.

- Output: a model that's helpful, honest, harmless.
- Cost: hundreds of thousands of human preference judgements.
- This is what makes a model helpful and harmless — and where the question of *whose* values get instilled begins.

RLHF — how it works

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK

1. Take an instruction-tuned model. Generate **two** responses per prompt.
2. A human rater picks the better one.
3. Train a *reward model* to predict the human's choice.
4. Use that reward model as the reward signal in **PPO** (a reinforcement learning algorithm) to fine-tune the original LLM.
5. The LLM updates towards generating responses the reward model scores highly.



Warning

Reward hacking is a critical issue with RLHF. The LLM learns to *game* the reward model—producing responses that *look* better without being better.

- Detecting and preventing this is a lot of what constitutes “alignment research”

Reward hacking — you get what you reward

The model optimises what raters *reward*. That is not the same as what raters *want*.

Raters, on average:

- favour **longer** answers → models get wordy.
- favour **agreement** → models get sycophantic — they tell you what you already believe.
- favour a **confident tone** → models state guesses as facts.
 - Cf. current issues with Opus 5



Warning

Goodhart's law: when a measure becomes a target, it stops being a good measure.

Reward hacking — a miniature

Q: What is the capital of Australia?

Answer A — hedged, correct

"I believe it is Canberra — though Sydney and Melbourne are the cities most people guess."

Answer B — confident, wrong

"Sydney is the capital of Australia. As the country's largest city and its economic centre, Sydney has been the seat of national life since federation."

A reward model that has learned to favour length and confidence scores **B higher**. The correct answer loses.

THREE PARTIAL FIXES

- **Better rater instructions** — tell raters to check facts and not to reward padding.
- **Penalising length** — subtract a length term from the reward (a form of indirect regularisation)
- **DPO's implicit reward** (next slide) — no separate reward model to over-optimize against; the rater biases stay in the preference data, though.

DPO — the simpler alternative

DIRECT PREFERENCE OPTIMIZATION, RAFAILOV ET AL. 2023

DPO skips the explicit reward model.

- Given (prompt, preferred response, dispreferred response) triples.
- Optimise the LLM **directly** to make the preferred response more likely.
- No reinforcement learning. No PPO. No reward model.

Mathematically equivalent to RLHF in the limit, but simpler.

Many recent open models (Llama 3, Qwen, Mistral) use DPO or a variant.

Anthropic's **Constitutional AI** is another approach — use the *model itself* (or a separate model) to critique and revise responses, then fine-tune on the revisions.

Context windows — how much can the model read?

The **context window** is the maximum number of tokens the model can attend over in one pass — prompt and response together.

Cost is the constraint: attention compute grows with the **square** of length — yesterday's $\mathcal{O}(T^2)$ slide, back again. Double the window, quadruple the compute.

2019

GPT-2: **1,024** tokens.

At ~ 0.75 words per token, that is $1,024 \times 0.75 \approx 770$ words — about two pages.

2025

Production models reach $\sim$ **2M** tokens; Llama 4 Scout claims **10M**.

2M tokens $\approx$ 1.5 million words — a few thousand pages.

Lost in the middle

A long window is not the same as a well-used one.

Liu et al. ([2024](#)) placed one needed fact at different positions inside a long context and measured whether models answered with it:

- Fact at the **start** → high accuracy.
- Fact at the **end** → high accuracy.
- Fact in the **middle** → accuracy dips. The curve is **U-shaped**.



Retrieval

Dumping every document you own into a huge window underperforms **retrieving the right passage** and placing it in the prompt.

RAG gives the model an “open-book” exam

≈ 3 MIN

A base LLM answers from memory alone — and **hallucinates** when it doesn't know.
You have a database of 10,000 company documents and the ability to embed any document.

TRY

How would you stop the model inventing answers about those 10,000 documents — *without retraining it?*

RETRIEVAL-AUGMENTED GENERATION (RAG)

1. **Embed the user's question** (Day 8!).
2. **Search** the database for the most similar documents.
3. **Inject** those documents into the prompt as context.
4. The model answers *from the supplied text*, not just memory.

Used by ChatGPT browsing, Perplexity, and most enterprise LLM products. No retraining needed. **The hard part is retrieval** — getting the *right* documents into context is itself a hard problem.

Two ways to make a base model do more

BOTH BOLT ONTO THE MODEL YOU BUILT — NO NEW ARCHITECTURE

Reach outward — tools & agents

Give the model a **menu of tools** it may invoke: web search, run Python, query a database, call any API you describe.

The model decides *when* to call a tool and *what* to pass; the output is fed back as context.

This is the foundation of the **AI agents** push of 2024–25 — systems that book travel, write and run code, file pull requests

- We can watch as this becomes ever more ubiquitous

Think longer — reasoning models

Teach the model to **think out loud** before answering — OpenAI's **o1**, Claude's **extended thinking**, DeepSeek's **R1**.

It generates hidden “chain-of-thought” tokens, then answers.

Trained by **reinforcement learning on outcomes** — rewarded for getting answers *right*, not for any particular trace.

- Starting to make *real* in-roads on mathematical problems, among others

LoRA — fine-tuning without touching the base model

Full fine-tuning updates **every** weight matrix in the network. For a 7B-parameter model that is 7 billion numbers to update and store — per task.

LoRA (low-rank adaptation; Hu et al., 2021) trains far fewer:

Take one weight matrix $\mathbf{W}$, of size $d \times d$, where d is the layer width.

- **Freeze $\mathbf{W}$** — it never changes.
- Learn a correction $\Delta\mathbf{W} = \mathbf{BA}$, where $\mathbf{A}$ is an $r \times d$ matrix and $\mathbf{B}$ is $d \times r$.
- r is the **rank** of the correction — chosen small, $r \ll d$.
- The layer now uses $\mathbf{W} + \mathbf{BA}$; only $\mathbf{A}$ and $\mathbf{B}$ are trained.

The product $\mathbf{BA}$ is $(d \times r)(r \times d) = d \times d$ — the same shape as $\mathbf{W}$, built from far fewer numbers.

LoRA — the arithmetic

Take one 768×768 weight matrix (GPT-2's width):

$$\begin{aligned} \text{full matrix} &: 768 \times 768 = 589,824 \text{ weights} \\ \text{LoRA, } r = 8 &: \underbrace{8 \times 768}_A + \underbrace{768 \times 8}_B = 6,144 + 6,144 = 12,288 \\ \text{ratio} &: 589,824 \div 12,288 = 48 \times \text{ fewer} \end{aligned}$$

No new biases enter the count — LoRA adds only the two matrices, and the frozen layer keeps its own bias.
Applied to every attention matrix in the model, the trainable share is typically **under 1%** of the parameters.

LoRA — your turn

TRY IT YOURSELF

A rank-16 LoRA on a $1,024 \times 1,024$ matrix. How many trainable parameters, and how many times fewer than training the full matrix?

ANSWER

A: $16 \times 1,024 = 16,384$. **B:** $1,024 \times 16 = 16,384$. Total: $2 \times 16 \times 1,024 = 32,768$.

Full matrix: $1,024 \times 1,024 = 1,048,576$. Ratio: $1,048,576 \div 32,768 = 32$.



Tip

Why people do this: **one frozen base model, plus a small adapter file per task**. Swap the adapter and you swap the behaviour — no full model copy each time.

Multimodal extensions

NOT JUST TEXT

Vision-language models

- GPT-4V, Claude 3 Vision, Gemini.
- Images get **patchified** — cut into a grid of squares — then embedded and fed in alongside text tokens.
- Same architecture; one new tokeniser-equivalent for pixels. Next slide.

Audio + video

- Whisper for speech-to-text.
- GPT-4o native audio (sub-second latency).
- Sora, Veo for video generation.
- All variations on the transformer architecture.



The transformer is modality-agnostic

Anything that can be turned into a sequence of vectors — text, image patches, audio frames, video frames, even amino-acid residues — can be fed through it.

This is why “what is a transformer” matters: it is the sequence-modelling primitive underneath all of them.

How does an image become tokens?

PATCHIFYING — THE TOKENISER FOR PIXELS

A transformer takes a sequence of vectors. An image isn't one in the same sense as text, so we build one:

1. Cut the image into a grid of fixed-size squares. **16×16 pixels** is the standard.
 2. Flatten each patch — $16 \times 16 \times 3 = 768$ numbers.
 3. Project it to the model's width d with a learned linear layer.
 4. That vector is a token.
- Add a positional embedding and feed it in.

Why not one token per pixel?

A 224×224 image, straight in: $T = 50,176$.

Cut into 16×16 patches: $(224/16)^2 = 196$ patches, so $T = 196$.

256× shorter — and attention is quadratic, so about **65,000× less attention compute**.

The same knob, again

Patch size does for pixels what tokens-per-word does for text. Bigger patches, shorter sequence, cheaper attention, coarser detail.

Dosovitskiy et al. (2020) — *An Image is Worth 16×16 Words*. The title is the whole idea.

Making transformers cheaper to run

THE EFFICIENCY FRONTIER — SAME IDEA, FAR LESS COMPUTE

Longer context, cheaper attention

- **FlashAttention** — memory-efficient tiling. Still $O(T^2)$, just smaller constants. Now standard.
- **Sparse / sliding attention** — attend to a local window; recover global reach by stacking layers (Longformer).
- **State-space models (Mamba)** — replace attention with a learned recurrence. $O(T)$ in sequence length.

Smaller, faster models

- **Quantisation** — lower-precision weights (fp16 → int8 → int4). 4–8× smaller; slight quality loss.
- **Distillation** — train a small model to mimic a big one (Llama 3.2 1B from Llama 3 70B).
- **Speculative decoding** — a small model proposes tokens; the big one verifies in batch. 2–3× faster, no quality loss.
- **Mixture-of-experts (MoE)** — route each token to a few “expert” sub-networks: huge total size, small *active* cost.



Tip

The shift since 2023: matching frontier quality at a fraction of the cost. The cheapest API of 2025 outperforms the most expensive API of 2023. This course gives you the **vocabulary** to read any of these papers without getting lost.

Quantisation — the memory ledger

The last slide said quantised models are “4–8× smaller”. Here is where that comes from. Memory = number of parameters × bytes per parameter. For a 7-billion-parameter model:

Precision	Bytes per weight	Memory
fp32	4	$7\text{B} \times 4 = 28\text{ GB}$
fp16	2	$7\text{B} \times 2 = 14\text{ GB}$
int8	1	$7\text{B} \times 1 = 7\text{ GB}$
int4	0.5	$7\text{B} \times 0.5 = 3.5\text{ GB}$



Tip

28 GB needs a data-centre GPU. **3.5 GB fits in an ordinary laptop’s memory** — the same model, one-eighth the size. This is why you can run a 7B model at home!

Quantisation — what you pay

Each weight is **rounded to a coarser grid**:

- int8 stores each weight as one of $2^8 = 256$ values.
 - int4 stores each weight as one of $2^4 = 16$ values.
-
- Quality falls little down to 4 bits; below that it falls apart.
 - In the common case this is **post-training quantisation**: no retraining. Take the trained weights and round them, choosing the grid per group of weights so the large ones survive.



Note

Sixteen values per weight sounds destructive. It works because the model's behaviour is spread across billions of weights — no single weight needs to be precise.

Open vs closed-source LLMs

A CONSEQUENTIAL SPLIT

Closed (API only)

GPT-5, Claude Opus 4.7, Gemini 2.5.

- Highest quality.
- Pay per token.
- Cannot inspect weights.
- Cannot fine-tune (mostly).
- Cannot run offline.

Open-weights

Llama 4, Qwen 3, Mistral, DeepSeek, Gemma.

- Quality has closed the gap (within ~1 year of frontier).
- Free to download and run.
- Can fine-tune for your data.
- Can run on local hardware.
- Can audit (partially) what the model knows.



Tip

For research and reproducibility, open-weights models are increasingly attractive. For *peak* performance, closed APIs still win. For most applications in 2025+, the open models are *good enough* — and the math of inference cost favours them at scale.

Take stock

WHERE YOU ARE AT THE END OF DAY 11

You can:

1. Define and tokenise text with BPE.
2. Explain why subword tokenisation has won.
3. Sample from a language model with multiple strategies (greedy, top-k, top-p, temperature).
4. Build, train, and sample from a small GPT — start to finish.
5. Name the post-training steps (SFT, RLHF, DPO) that turn a base model into a chat product.
6. Place the rest of the modern stack — RAG, tools, reasoning, multimodal, efficiency tricks — on a map relative to the GPT you built.
7. Articulate at least one open social-science research question about LLMs as instruments.

This is the conceptual core of the modern AI landscape.

| Lab brief

Today's lab — 90 minutes

GOALS

Take yesterday's GPT and polish it:

1. Replace the character tokeniser with **BPE** (use `tiktoken` or hand-rolled).
2. Re-tokenise the training corpus.
3. Re-train (model definition is unchanged — only the embedding vocab size differs).
4. Try **different sampling strategies** — greedy, top-k, top-p, varying temperature — on the same prompt.
5. **Side-by-side comparison** with yesterday's character-level output.

DELIVERABLE

A short comparison document:

- Three sample outputs (char-level GPT, BPE-GPT at $\tau = 0.5$, BPE-GPT at $\tau = 1.2$).
- One paragraph: "What changed, and what does it tell me about the model?"

Stretch: try a non-English corpus. Watch what happens to the token count.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.